

Tema 3.6: Seguridad en el Manejo de Sesiones y JSON Web Tokens (JWT)

Resumen

Este tema aborda la evolución de la gestión de sesiones en aplicaciones web, comparando el enfoque tradicional basado en estado (*Stateful*) mediante cookies de servidor frente a la arquitectura descentralizada sin estado (*Stateless*) mediante **JSON Web Tokens (JWT)**. Se analiza la anatomía del estándar RFC 7519, las técnicas de prevención de secuestro de sesión (*Session Hijacking*), XSS y CSRF, y la implementación paso a paso de un proveedor de tokens y un filtro de autenticación personalizado en Spring Boot 3.x.

Concepto Principal (El 'Qué')

En el desarrollo de aplicaciones web y APIs RESTful, la **Gestión de Sesiones** representa el mecanismo mediante el cual el servidor mantiene el contexto de un cliente a través de múltiples solicitudes HTTP (un protocolo inherentemente sin estado o *stateless*).

1. Enfoque Basado en Estado (Stateful / Cookies de Servidor)

En aplicaciones tradicionales (MPA), el servidor genera un identificador único de sesión (JSESSIONID) y almacena los datos del usuario autenticado en la memoria del servidor (o en una caché distribuida como Redis). El identificador se envía al cliente en una cookie HTTP. * **Desventajas:** Dificulta el escalado horizontal (requiere *sticky sessions*), alto consumo de memoria en el servidor y vulnerabilidad a ataques **CSRF** (*Cross-Site Request Forgery*).

2. Enfoque Sin Estado (Stateless / Token-Based Authentication)

En arquitecturas modernas desacopladas (SPA y Microservicios), el servidor **no almacena ningún estado de sesión en memoria**. Al autenticarse exitosamente, el servidor emite un token firmado criptográficamente (**JSON Web Token - JWT**) que contiene los datos de identidad y permisos del usuario. El cliente almacena este token y lo envía en el encabezado de autorización HTTP

(Authorization: Bearer <token>) en cada solicitud posterior.

Anatomía de un JSON Web Token (RFC 7519)

Un JWT es una cadena de texto compacta dividida en tres partes delimitadas por puntos (.):

HEADER.PAYLOAD.SIGNATURE

1. **Header (Encabezado):** Indica el algoritmo de firma (ejemplo: HS256 o RS256) y el tipo de token ("typ": "JWT"). Codificado en Base64Url.
2. **Payload (Carga Útil):** Contiene los *Claims* (afirmaciones) sobre el usuario y metadatos. Existen claims registrados (sub - sujeto, iat - fecha emisión, exp - fecha expiración) y claims personalizados (roles, email). Codificado en Base64Url (¡no encriptado!).
3. **Signature (Firma):** Se calcula tomando el Header en Base64Url, el Payload en Base64Url, y procesándolos con una clave secreta compartida (*Secret Key*) mediante el algoritmo especificado. Previene la alteración o falsificación del token.

Analogía (La Intuición)

Imagine la diferencia entre una ficha de guardarropa en un teatro y un pasaporte digital biométrico:

- **Sesión Stateful (Ficha de Guardarropa):** Cuando usted entrega su abrigo, el guardarropa le da una ficha numérica de plástico (JSESSIONID). El abrigo físico se queda dentro del armario del teatro (memoria del servidor). Si usted pierde la ficha o el armario del teatro sufre un incendio (caída del servidor), no hay forma de verificar a quién pertenece el abrigo sin revisar registros en el sitio.
- **Sesión Stateless JWT (Pasaporte Digital Criptográfico):** Al ingresar al país, las autoridades le entregan una credencial digital portátil que contiene impresa de forma infalsificable su nombre, nacionalidad y permisos de estadía, firmada con el sello holográfico oficial del Ministerio (Firma Criptográfica). Usted lleva la credencial consigo a todas partes. Cualquier oficial en cualquier ciudad del país puede validar el sello criptográfico en tiempo real sin necesidad de llamar a la oficina central.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se implementa un sistema completo de tokens JWT en Java 21 con Spring Boot 3.x utilizando la librería jjwt:

1. Proveedor de Tokens JWT (JwtTokenProvider)

```
package cr.ac.ucr.paraíso.ie.expresofast.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.stream.Collectors;

@Component
public class JwtTokenProvider {

    private final SecretKey key;
    private final long jwtExpirationInMs;

    public JwtTokenProvider(
        @Value("${app.jwt.secret}") String secret,
        @Value("${app.jwt.expiration-ms}") long jwtExpirationInMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.jwtExpirationInMs = jwtExpirationInMs;
    }

    public String generarToken(Authentication authentication) {
        String username = authentication.getName();
        Date ahora = new Date();
        Date fechaExpiracion = new Date(ahora.getTime() + jwtExpirationInMs);

        String roles = authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.joining(","));

        return Jwts.builder()
            .setSubject(username)
            .setExpiration(fechaExpiracion)
            .claim("roles", roles)
            .signWith(key)
            .compact();
    }
}
```

```

        .subject(username)
        .claim("roles", roles)
        .issuedAt(ahora)
        .expiration(fechaExpiracion)
        .signWith(key)
        .compact();
    }

    public String obtenerUsernameDelJWT(String token) {
        Claims claims = Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload();
        return claims.getSubject();
    }

    public boolean validarToken(String token) {
        try {
            Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException ex) {
            // Log de token caducado, malformado o firma inválida
            return false;
        }
    }
}

```

2. Filtro de Autenticación de Peticiones (JwtAuthenticationFilter)

```

package cr.ac.ucr.paraíso.ie.expresofast.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

```

```

import java.io.IOException;

public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider,
                                   UserDetailsService userDetailsService) {
        this.tokenProvider = tokenProvider;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                     HttpServletResponse response,
                                     FilterChain filterChain)
        throws ServletException, IOException {

        String token = obtenerJwtDeLaSolicitud(request);

        if (StringUtils.hasText(token) && tokenProvider.validarToken(token)) {
            String username = tokenProvider.obtenerUsernameDelJWT(token);

            UserDetails userDetails = userDetailsService.loadUserByUsername(username);
            UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities()
                );
            authentication.setDetails(
                new WebAuthenticationDetailsSource().buildDetails(request)
            );

            SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
    }

    private String obtenerJwtDeLaSolicitud(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}

```

```
}
```

3. Registro del Filtro en la Seguridad de Spring (SecurityConfig)

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(sess -> sess
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated()
        )
        .addFilterBefore(
            new JwtAuthenticationFilter(tokenProvider, userDetailsService),
            UsernamePasswordAuthenticationFilter.class
        );

    return http.build();
}
```

Explicación Paso a Paso del Código:

1. **JwtTokenProvider**: Firma los tokens mediante la clave secreta `SecretKey` utilizando la especificación HMAC-SHA. En los *claims*, empaqueta el sujeto (`subject`), la fecha de expiración (`exp`) y los roles concedidos.
2. **JwtAuthenticationFilter**: Intercepta **todas** las peticiones HTTP entrantes extendiendo de `OncePerRequestFilter`. Extrae el token enviado en el encabezado `Authorization: Bearer <token>`, valida su integridad sintáctica y firma criptográfica.
3. **Poblado de SecurityContextHolder**: Si el token es válido, instancia un `UsernamePasswordAuthenticationToken` y lo establece en el contexto de seguridad actual. A partir de este momento, las anotaciones `@PreAuthorize` y las reglas de `SecurityFilterChain` reconocen al usuario como autenticado durante el ciclo de vida de esa petición.
4. **SessionCreationPolicy.STATELESS**: Le indica a Spring Security que **nunca** cree objetos `HttpSession` en el servidor ni guarde galletas de sesión en memoria.

Trampas Comunes

Confidencialidad Deficiente en el Payload de JWT:

- *Error:* Incluir contraseñas, números de tarjeta de crédito o datos médicos en el Payload del token creyendo que la firma los encripta.
- *Solución:* Recordar que el Payload únicamente está codificado en Base64Url (legible por cualquier persona que intercepte el token). Incluir únicamente identificadores no sensibles y autorizaciones básicas.

Claves Secretas Débiles o Hardcodeadas:

- *Error:* Utilizar claves secretas cortas (como "secret123") en archivos de código fuente subidos a repositorios públicos de Git.
- *Solución:* Generar claves secretas de al menos 256 bits (32 caracteres de alta entropía) y cargarlas mediante variables de entorno del sistema (`System.getenv()`).

Vulnerabilidad a XSS y Mal Almacenamiento en Cliente:

- *Error:* Almacenar el JWT en `localStorage` en aplicaciones vulnerables a inyecciones Cross-Site Scripting (XSS), permitiendo a scripts maliciosos robar el token.
- *Solución:* Sanitizar rigurosamente las entradas de usuario en el frontend o almacenar los tokens en cookies con banderas `HttpOnly`, `Secure` y `SameSite=Strict`.

Pregunta de Reflexión

Dado que los tokens JWT son stateless e infalsificables una vez firmados:

¿De qué manera una arquitectura de software puede revocar o cancelar inmediatamente el token JWT de un usuario cuya cuenta ha sido comprometida antes de que el token alcance su tiempo límite de expiración (exp)?